EXPERIMENT – 2

AIM:

Implement breadth-first search, depth-first search and uniform-cost search on graph or maze problems; compare search cost and path quality.

OBJECTIVE:

To implement and compare three fundamental uninformed search strategies — Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform-Cost Search (UCS) — on a graph, and to analyse the path quality and search cost produced by each strategy.

PROCEDURE:

1. Represent the problem as a directed graph using an adjacency list, with an additional weighted version of the same graph for UCS.
2. Implement Breadth-First Search using a FIFO queue that explores the graph level by level, guaranteeing the shortest path in terms of number of edges.
3. Implement Depth-First Search using a LIFO stack that explores as far as possible along each branch before backtracking.
4. Implement Uniform-Cost Search using a priority queue (min-heap) that always expands the node with the lowest cumulative path cost g(n).
5. Run all three algorithms from the start node 'A' to the goal node 'G' on the sample graph.
6. Compare the paths returned by BFS, DFS and UCS, and note the total path cost computed by UCS.
7. Tabulate the differences in traversal strategy, data structure used, and the quality of the resulting path/cost.

PROGRAM / CODE:

```
from collections import deque import
 heapq
 # ---------- Graph ----------
 graph = {
       "A": ["B", "C"],
       "B": ["D"],
       "C": ["D"],
       "D":["G"], "G": []
 }
 weighted_graph = {
       "A": [("B", 1), ("C", 5)],
       "B": [("D", 1)],
       "C": [("D", 1)],
       "D": [("G", 1)], "G": []
 }
 # ---------- BFS ----------
 def bfs(graph, start, goal): queue =
       deque([[start]]) visited = set()
       while queue:
             path = queue.popleft() node =
             path[-1]
              if node == goal: return
                    path
              if node not in visited: visited.add(node)
                    for neighbor in graph[node]:
                          queue.append(path + [neighbor])
 # ---------- DFS ----------
```

```
def dfs(graph, start, goal): stack =
    [[start]] visited = set()
    while stack:
        path = stack.pop() node =
        path[-1]
          if node == goal: return
              path
          if node not in visited: visited.add(node)
              for neighbor in graph[node]: stack.append(path
                  + [neighbor])
# -------------------- Uniform Cost Search def --------------------
ucs(graph, start, goal):
    pq = [(0, start, [start])] visited = set()
    while pq:
        cost, node, path = heapq.heappop(pq) if node ==
          goal:
              return cost, path if node
          not in visited:
              visited.add(node)
              for neighbor, weight in graph[node]:
                  heapq.heappush(pq, (cost + weight, neighbor, path + [neighbor])) #          Run
print("="*45) ------------      --------------------
print("          ARTIFICIAL INTELLIGENCE PRACTICAL")
print("          BFS | DFS | Uniform Cost Search")
print("="*45)
bfs_path = bfs(graph, "A", "G") dfs_path =
dfs(graph, "A", "G")
cost, ucs_path = ucs(weighted_graph, "A", "G") print("\nBFS
Path :", " -> ".join(bfs_path))
print("DFS Path :", " -> ".join(dfs_path))
print("UCS Path :", " -> ".join(ucs_path)) print("Total Cost:",
cost) print("\nComparison")
print("-"*45)
print("BFS : Uses Queue (FIFO) and finds shortest path.") print("DFS : Uses Stack (LIFO)
and explores deep first.") print("UCS : Uses Priority Queue and finds minimum cost path.")
```

OUTPUT:

```
=============================================
          ARTIFICIAL INTELLIGENCE PRACTICAL
          BFS | DFS | Uniform Cost Search
 ============================================= BFS
 Path : A -> B -> D -> G
 DFS Path : A -> C -> D -> G UCS Path :
 A -> B -> D -> G
 Total Cost: 3 Comparison
 -----------------------------------------------
 BFS : Uses Queue (FIFO) and finds shortest path. DFS : Uses
 Stack (LIFO) and explores deep first.
 UCS : Uses Priority Queue and finds minimum cost path.
```

RESULT:

BFS, DFS, and UCS were implemented on the same graph. BFS found the shortest path by edge count (A → B → D → G), while DFS returned a non-optimal path (A → C → D → G). UCS found the minimum-cost path (A → B → D → G, cost = 3), confirming that UCS is optimal for weighted graphs.